

Lenguaje Python

Cursada 2026

Clase 9: más sobre la POO en Python

Repasemos algunos conceptos

Conceptos claves: objetos y clases

- La **clase** define las variables de instancia y los métodos. Permite agrupar objetos con características similares.
- Los **objetos** son instancias de una clase.
- Cuando se crea un objeto, se ejecuta automáticamente el método `__init()` que permite **inicializar** el objeto.
- La definición de la clase especifica qué partes son públicas y **cuáles vamos a considerar no públicas**.

¿Cómo se especifica **privado o público** en Python?

Mensajes y métodos

TODO el procesamiento en este modelo es activado por mensajes entre objetos.

- El **mensaje** es el modo de comunicación entre los objetos. Cuando se invoca una función de un objeto, lo que se está haciendo es **enviando un mensaje** a dicho objeto.
- El **método** es la función que está asociada a un objeto determinado y cuya ejecución sólo puede desencadenarse a través del envío de un mensaje recibido.

¿@property?

■ `@property` permite crear **propiedades** de una clase.

- Se aplica la función **property()** cuya forma general es:

```
property(fget=None, fset=None, fdel=None, doc=None)
```

¿En qué casos podemos usar propiedades?

Ejemplo

```
In [ ]: class Demo:
        def __init__(self):
            self._x = 0

        def get_x(self):
            print('Estoy en get')
            return self._x

        def set_x(self, value):
            print('Estoy en set')
            self._x = value

        x = property(get_x, set_x)
```

```
In [ ]: obj = Demo()
        obj.x = 10
        print(obj.x)
```

O en su forma equivalente ...

```
In [ ]: class Demo:
        def __init__(self):
            self._x = 0
        @property
        def x(self):
            print('Estoy en get')
            return self._x

        @x.setter
        def x(self, value):
            print('Estoy en set')
            self._x = value
```

```
In [ ]: obj = Demo()
        print(obj.x)
```

- @property es un **decorador**.

¿Qué es un decorador?

Un **decorador es una función** que recibe una función como argumento y **extiende** el comportamiento de esta última función sin modificarla explícitamente.

Revisemos el material que dejamos.

Hay otros decoradores

Métodos de clase

¿A qué creen que hacen referencia?

Corresponden a los mensajes que se envían a la **clase**, no a las instancias de la misma.

- Se utiliza el decorador **@classmethod**.
- Se usa **cls** en vez de **self**. ¿A qué hace referencia este argumento?

```
In [ ]: class Band():
        """ Define la entidad que representa a una banda .. """
        all_genres = set()

        @classmethod
        def clean_genres(cls, confirm=False):
            if confirm:
                cls.all_genres = set()

        def __init__(self, name, genres="rock"):
            self.name = name
            self.genres = genres
            self._members = []
            Band.all_genres.add(genres)

        def add_member(self, new_member):
            self._members.append(new_member)
```

```
In [ ]: soda = Band("Soda Stereo")
        nompa = Band("Nonpalidece", genres="reggae")
```

```
In [ ]: Band.all_genres
```

```
In [ ]: Band.clean_genres(True)
```

Métodos estáticos

Son funciones definidas dentro de una clase que **no acceden ni modifican el estado de la instancia (self) ni de la clase (cls)**.

- Se utiliza el decorador **@staticmethod**.
- NO se usa ni **cls** ni **self** ya que no reciben argumentos automáticos y se utilizan para funciones "útiles" que lógicamente pertenecen a la clase pero no corresponden al comportamiento de los objetos.

Veamos esto en un ejemplo

```
In [ ]: class Band():
        """ Define la entidad que representa a una banda """

        def __init__(self, name, genres="rock"):
            # Usamos el método estático para normalizar el nombre al nacer
            self.name = self.format_name(name)
            self.genres = genres
            self._members = []

        def add_member(self, new_member):
            self._members.append(new_member)

        # --- MÉTODO ESTÁTICO ---
        @staticmethod
        def format_name(name):
            """
            Quita espacios extra y pone mayúsculas iniciales en el nombre de
            IMPORTANTE: notar que NO usa 'self' ni 'cls'.
            """
            return name.strip().title()
```

```
In [ ]: # 1. Uso sin crear una instancia (Utilidad pura)
        real_name = " the rolling stones "
        clean_name = Band.format_name(real_name)
        print(f"Nombre procesado: '{clean_name}'")
```

```
In [ ]: # 2. Uso dentro de la clase
        new_band = Band("rage Against the machine", "rock")
        print(f"Banda creada: {new_band.name}")
```

Hablemos de herencia

- Es uno de los conceptos más importantes de la POO.
- La herencia permite que una clase pueda *heredar* los atributos y métodos de otra clase, que se **agregan** a los propios.
- Este concepto permite sumar, es decir **extender** una clase.
- La clase que hereda se denomina **clase derivada** y la clase de la cual se deriva se denomina **clase base**.

Observemos estas clases

```
In [ ]: class Vehículo:
    def __init__(self, cantidad_puertas, cantidad_ruedas, motor='Diesel'):
        self.cant_puertas = cantidad_puertas
        self.cant_ruedas = cantidad_ruedas
        self.motor = motor

    def __str__(self):
        return (f'Motor: {self.motor}, cantidad puertas: {self.cant_puert
```

```
In [ ]: class Automóvil(Vehículo):
    def __init__(self, marca, modelo, patente, puertas, ruedas, motor):
        Vehículo.__init__(self, puertas, ruedas, motor)
        self.marca = marca
        self.modelo = modelo
        self._patente = patente

    @property
    def patente(self):
        return self._patente

    @patente.setter
    def patente(self, nueva_patente):
        self._patente = nueva_patente

    def __str__(self):
        return (f'El auto {self.marca} {self.modelo}, patente: {self._pat
```

- ¿Cuál es la clase base? ¿Y la clase derivada? ¿Cuáles son las variables de instancia de un objeto Automóvil?
- ¿Por qué invocamos a **Vehículo__init__()**? ¿Qué pasa si no hacemos esto?

```
In [ ]: auto_nuevo = Automóvil('FIAT', "Cronos", 'AA588ZM', 5, 4, 'Nafta')
print(auto_nuevo)
```

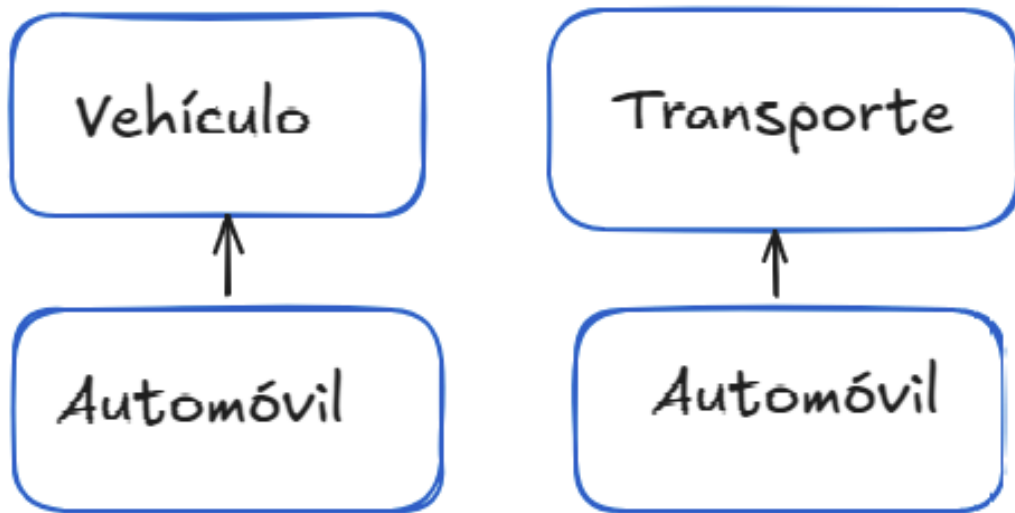
Si trabajamos con vehículos, también podemos definir ...

```
In [ ]: class Transporte():
    def __init__(self, max_pasajeros, tipo='Público'):
        self.tipo = tipo
        self.cant_max_pasajero = max_pasajeros

    def __str__(self):
        return (f'Tipo de transporte: {self.tipo}')
```

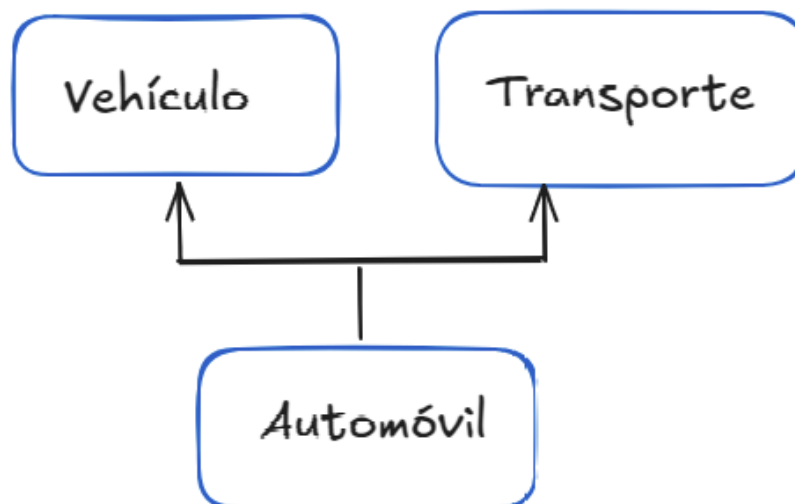
Un Auto es un vehículo, pero también es un medio de transporte ...

Podríamos pensar en algo así:



Aunque en Python podemos hacer algo mejor ...

Python tiene herencia múltiple



- Un automóvil es **"es un"** vehículo, pero también **es un** transporte.

En Python ...

```
In [ ]: class Automóvil(Vehículo, Transporte):
    def __init__(self, marca, modelo, patente, tipo='Particular', pasajero):
        Vehículo.__init__(self, puertas, ruedas, motor)
        Transporte.__init__(self, pasajeros, tipo)
        self.marca = marca
        self.modelo = modelo
        self._patente = patente

    @property
    def patente(self):
```

```

        return self._patente

    @patente.setter
    def patente(self, nueva_patente):
        self._patente = nueva_patente

```

```

In [ ]: auto_nuevo = Automóvil('FIAT', "Cronos", 'AA588ZM', 'Particular', puertas)
print(auto_nuevo)

```

A tener en cuenta ...

- MRO "Method Resolution Order"
- Por lo tanto, es MUY importante **el orden en que se especifican las clases bases**.
- Más información en [documentación oficial](#)

```

In [ ]: Automóvil.__mro__

```

```

In [ ]: class Automóvil(Transporte, Vehículo):
    def __init__(self, marca, modelo, patente, tipo='Particular', pasajeros=0, motor=False):
        Vehículo.__init__(self, puertas, ruedas, motor)
        Transporte.__init__(self, pasajeros, tipo)
        self.marca = marca
        self.modelo = modelo
        self._patente = patente

    @property
    def patente(self):
        return self._patente

    @patente.setter
    def patente(self, nueva_patente):
        self._patente = nueva_patente

```

```

In [ ]: Automóvil.__mro__

```

DEASAFÍO 1

¿Cuál es el MRO de D?

```

In [ ]: class A():
    pass
class B(A):
    pass
class C(A):
    pass
class D(B, C):
    pass

```

```

In [ ]: D.__mro__

```

Hagamos el siguiente cambio

```
In [ ]: class Vehículo():
        def __init__(self, cantidad_puertas, cantidad_ruedas, motor='Diesel'):
            self.cant_puertas = cantidad_puertas
            self.cant_ruedas = cantidad_ruedas
            self.motor = motor
            self.tipo = 'Sedan'
```

```
In [ ]: class Transporte():
        def __init__(self, max_pasajeros, tipo='Público'):
            self.tipo = tipo
            self.cant_max_pasajero = max_pasajeros

        def __str__(self):
            return (f'Tipo de transporte: {self.tipo}')
```

¿Cuál es el problema?

```
In [ ]: class Automóvil(Transporte, Vehículo):
        def __init__(self, marca, modelo, patente, tipo='Particular', pasajero):
            Transporte.__init__(self, pasajeros, tipo)
            Vehículo.__init__(self, puertas, ruedas, motor)

            self.marca = marca
            self.modelo = modelo
            self.patente = patente
        def __str__(self):
            return (f'El {self.marca} {self.modelo} es un {self.tipo} de {self.pasajero}')
```

```
In [ ]: auto_nuevo = Automóvil('FIAT', "Cronos", 'AA588ZM', 'Particular', 5)
        print(auto_nuevo)
```

¿Qué tenemos que tener en cuenta?

- El orden de las clases bases en la definición de la clase.
- El orden de la invocación de los `__init__`.

CONSIGNA: teníamos algo que investigar

¿Qué podemos decir de las variables de instancias cuyo nombre comienza con `__`?

[+Info en la documentación oficial](#)

El Name Mangling

Es un mecanismo para evitar colisiones de nombres en la herencia.

¿Qué era esto?

Cuando Python ve un atributo llamado **__atributo** dentro de una clase, lo renombra internamente siguiendo esta fórmula:

`_NombreDeLaClase__nombre_del_atributo`

- Recordemos que esto no implica que la variable es privada.

¿Cuándo se aplica?

- Nombres que comienzan con dos "_" y terminen con ningún o un único guión bajo.
- ¿Por qué esto no aplica a `__init__`, `__str__` etc.?

Habíamos visto este ejemplo

```
In [ ]: class SecretCode:
        """¿¿¿Textos con clave??? """

        def __init__(self, plain_text, secret_key):
            self.__plain_text = plain_text
            self.__secret_key = secret_key

        def decrypt(self, secret_key):
            """Solo se muestra el texto si secret_key es correcta"""

            if secret_key == self.__secret_key:
                return self.__plain_text
            else:
                return ''
```

```
In [ ]: secret_text = SecretCode('Seminario Python', 'stark')
```

```
In [ ]: print(secret_text.__plain_text)
```

```
In [ ]: print(secret_text._SecretCode__plain_text)
```

¿Para qué se usa entonces?

- La razón principal no es esconder datos del usuario, sino proteger a las subclases.
- Observemos el siguiente ejemplo:

```
In [ ]: class Vehículo:
        def __init__(self, cantidad_puertas, cantidad_ruedas, motor='Diesel'):
            self.cant_puertas = cantidad_puertas
            self.cant_ruedas = cantidad_ruedas
            self.motor = motor
            # Identificador interno de fábrica (Mangling)
            self.__id_interno = "FAB-12345"

        def __str__(self):
            return f'Motor: {self.motor}, puertas: {self.cant_puertas}, rueda
```

```

class Transporte():
    def __init__(self, max_pasajeros, tipo='Público'):
        self.tipo = tipo
        self.cant_max_pasajero = max_pasajeros
        # Identificador interno administrativo (Mangling)
        self.__id_interno = "ADM-99999"

    def __str__(self):
        return f'Tipo de transporte: {self.tipo}'

```

```

In [ ]: class Automóvil(Vehículo, Transporte):
        def __init__(self, marca, modelo, patente, tipo='Particular', pasajero):
            Vehículo.__init__(self, puertas, ruedas, motor)
            Transporte.__init__(self, pasajeros, tipo)
            self.marca = marca
            self.modelo = modelo
            self._patente = patente

```

```

In [ ]: auto = Automóvil("Toyota", "Corolla", "ABC-123")

# Da error al intentar acceder directamente
#print(auto.__id_interno)

# Se puede acceder a ambos atributos "mangled"
print("Atributos del Automóvil:")
print(f"ID de Fábrica: {auto._Vehículo__id_interno}")
print(f"ID Administrativo: {auto._Transporte__id_interno}")

```

Un último ejemplo: esto aplica también a métodos

```

In [ ]: class Transporte():
        def __init__(self, max_pasajeros, tipo='Público'):
            self.tipo = tipo
            self.cant_max_pasajero = max_pasajeros
            self.__id_interno = "ADM-99999"

        def __str__(self):
            return f'Tipo de transporte: {self.tipo}'

        def _validar_capacidad(self, actuales):
            """Lógica para saber si suben más pasajeros."""
            return actuales <= self.cant_max_pasajero

        def _actualizar_seguro_transporte(self):
            """Trámite interno de la administración."""
            print(f"Actualizando póliza para transporte tipo {self.tipo}...")

```

¿Qué diferencias hay entre los métodos `_validar_capacidad` y `_actualizar_seguro_transporte`?

Al igual que con las variables de instancia:

- **metodo():** accesible desde cualquier lugar (dentro y fuera de la clase)

- **`_metodo()`**: indica uso interno. Por convención se asume privado.
- **`__metodo()`**: Name Mangling: Python renombra el método a `_Clase__metodo` para evitar colisiones en herencia.

¿Qué términos asociamos con la programación orientada a objetos?

Destacados ...

- Encapsulamiento
 - **`class`**, métodos privados y públicos, propiedades.
- Herencia
 - Clases bases y derivadas.
 - Herencia múltiple.
- ¿Alguno más?

Polimorfismo

- Capacidad de los objetos de distintas clases de responder a mensajes con el mismo nombre.
- Ejemplo: + entre enteros y cadenas.

```
In [ ]: print("hola " + "que tal.")
        print(3 + 4)
```

Observemos este código

```
In [ ]: class Musician:
        def __init__(self, name, instrument=None, band=None):
            self.name = name
            self.has_a_band = band!=None
            self._band = band
            self.instrument = instrument

        @property
        def band(self):
            if self.has_a_band:
                return self._band
            else:
                return None

        @band.setter
        def band(self, new_band):
```

```
self._band = new_band
self.has_a_band = self._band!=None
```

¿Podemos sumar dos músicos?

```
In [ ]: adele = Musician("Adele")
        sting = Musician("Sting", "The Police")

        print(adele + sting)
```

No podemos sumar dos músicos, salvo que ...

```
In [ ]: class Musician:
        def __init__(self, name, instrument=None, band=None):
            self.name = name
            self.has_a_band = band!=None
            self._band = band
            self.instrument = instrument

        def __add__(self, other):
            return (f"Nuevo dúo: {self.name} y {other.name}")

        @property
        def band(self):
            if self.has_a_band:
                return self._band
            else:
                return None

        @band.setter
        def band(self, new_band):
            self._band = new_band
            self.has_a_band = self._band!=None
```

```
In [ ]: adele = Musician("Adele")
        sting = Musician("Sting", "The Police")

        print(adele + sting)
```

Creamos iteradores

Un **iterador** es un objeto que permite recorrer **uno a uno** los elementos de una estructura de datos para poder operar con ellos.

- Un objeto iterable tiene que implementar un **método** `__next__` que debe devolver los elementos, **de a uno por vez**, comenzando por el primero.
- Y al llegar al final de la estructura, debe levantar la **excepción** **StopIteration**.

- Todas implementan un método especial denominado `__iter__`.
 - `__iter__` devuelve un iterador capaz de recorrer la secuencia.

Los siguientes códigos son equivalentes:

```
In [ ]: lista = ['uno', 'dos', 'tres']
        for palabra in lista:
            print(palabra)
```

```
In [ ]: iterador = iter(lista)
        while True:
            try:
                palabra = next(iterador) # o iterador.__next__()
            except StopIteration:
                break
            print(palabra)
```

- La función `iter` retorna un objeto iterador.

¿Son equivalentes?

```
In [ ]: for elem in lista:
        print (elem)
```

```
In [ ]: for elem in iterador:
        print (elem)
```

Veamos este ejemplo:

```
In [ ]: class CadenaInvertida:
        def __init__(self, cadena):
            self._cadena = cadena
            self._posicion = len(cadena)

        def __iter__(self):
            return(self)

        def __next__(self):
            if self._posicion == 0:
                raise(StopIteration)
            self._posicion = self._posicion - 1
            return(self._cadena[self._posicion])
```

```
In [ ]: cadena_invertida = CadenaInvertida('La clase próxima empezamos con pandas')

        for caracter in cadena_invertida:
            print(caracter, end=' ')
```

- ¿Qué creen que imprime?

DESAFIO 2

Implementar la clase **CadenaCodificada**, que dada una cadena de caracteres, me permita trabajar con la misma en forma codificada, según la codificación Cesar. Podemos recorrer con un **for** un objeto de clase **CadenaCodificada**, lo cual permite acceder uno a uno a los caracteres codificados de la misma.

Ejemplo: si la cadena original es "Hola que tal", la cadena codificada será "lpmb!rvf!ubm"

Nota: implementar este objeto como un objeto iterable.

```
In [ ]: #Una posible solución que no vamos a mostrar pero si a ejecutar como para
from functools import reduce

class CadenaCodificada:
    def __init__(self, cadena):
        self._cadena = cadena
        self._posicion = 0

    def __iter__(self):
        return(self)

    def __next__(self):
        if self._posicion == len(self._cadena):
            raise(StopIteration)
        car = self._cadena[self._posicion]
        self._posicion = self._posicion + 1
        return(chr(ord(car) + 1))

    def __str__(self):
        lista = map(lambda c : chr(ord(c) + 1), self._cadena)
        return reduce(lambda c1, c2: c1 + c2, lista)
```

Ejemplo de uso de una CadenaCodificada

```
In [ ]: mi_cadena = CadenaCodificada("Hola que tal")

for caracter in mi_cadena:
    print(caracter, end=" ")
```

```
In [ ]: print(mi_cadena)
```

Para los que quieran seguir un poco más

...

- <https://realpython.com/python-classes/>
- <https://realpython.com/inheritance-composition-python/>

- <https://realpython.com/instance-class-and-static-methods-demystified/>

Seguimos la próxima ...